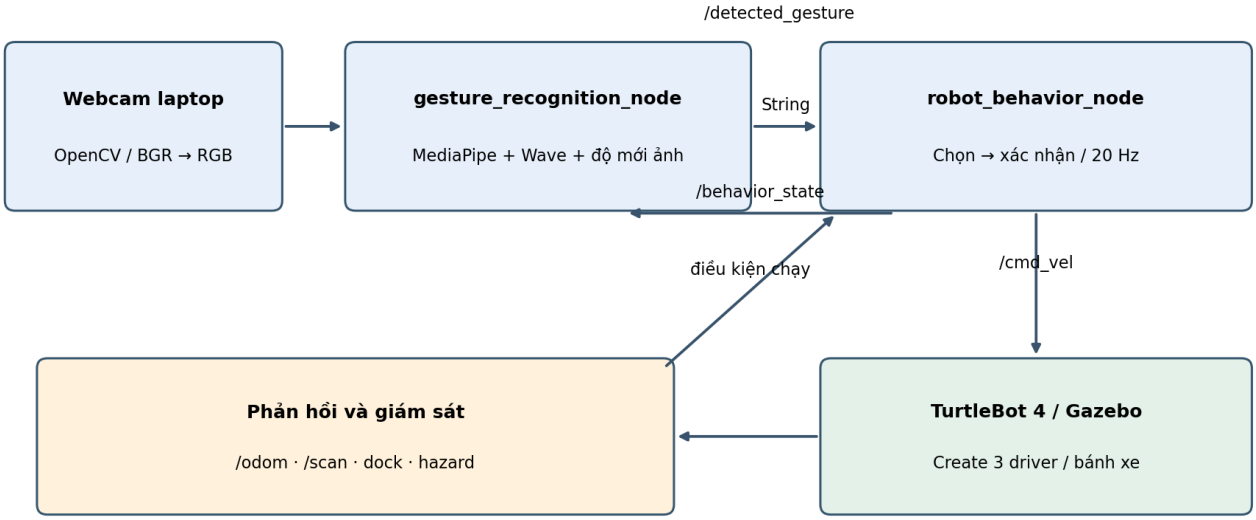
BÀI TẬP 6

Giao tiếp phi ngôn ngữ và điều khiển TurtleBot 4 bằng cử chỉ tay

Tô Văn Chúc · MSSV 23020727
Lớp: chờ bổ sung
Ubuntu 22.04 · ROS 2 Humble · Gazebo Fortress

1. Mục tiêu và kiến trúc hệ thống

Xây dựng hai ROS 2 node để nhận diện tay từ webcam laptop và điều khiển robot trong vùng trống. Người dùng chọn hành vi bằng cử chỉ, xác nhận bằng ngón cái; robot phản hồi trạng thái trên cửa sổ webcam. Bài này sử dụng điều khiển vận tốc và phản hồi cảm biến.



Hình 1. Luồng ảnh, nhãn cử chỉ, vận tốc và phản hồi của hệ thống.

Node	Nhiệm vụ
gesture_recognition_node	OpenCV đọc webcam; MediaPipe phân loại; lọc nhãn; hiển thị ảnh và trạng thái.
robot_behavior_node	Chọn–xác nhận; phát Twist ở 20 Hz; đo tiến trình bằng odometry; giám sát tín hiệu và cảm biến.

Source độc lập trong package **turtlebot4_gesture_hri**. Các file control.py và wave.py tách logic khỏi ROS để kiểm thử; behavior_node.py và recognition_node.py nối logic với topic, camera và giao diện.

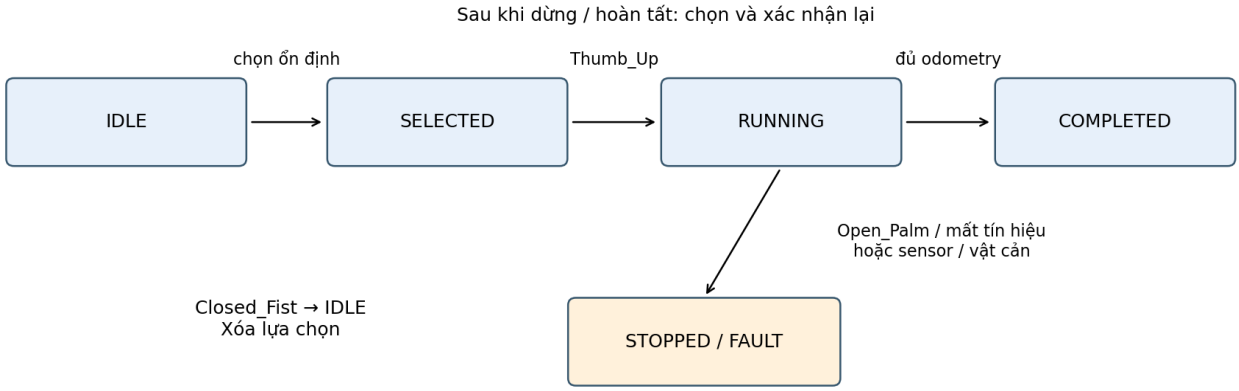
2. Nhận diện AI và ánh xạ cử chỉ

OpenCV lấy frame mới nhất, mirror ảnh và đổi BGR sang RGB. MediaPipe Tasks chạy VIDEO mode, cung cấp 21 điểm khớp AI và nhãn/confidence. Luật góc khớp, duỗi/gập ngón bổ sung khi model trả None hoặc nhãn mâu thuẫn. Giao diện ghi riêng Model và Detect, giữ xác suất model thô, không gán xác suất AI giả cho luật. Hai bàn tay hợp lệ hoặc tay bị cắt/không thấy đều vô hiệu điều khiển.

Cử chỉ / String	Ý nghĩa và chuyển động
Vẫy tay / Wave	Chọn chào: quay +45°, −90°, +45°.
Một ngón / Pointing_Up	Tiến 0.50 m rồi lùi về điểm đầu, lặp 4 chu kỳ. Tổng 8 đoạn.
Hai ngón / Victory	Chọn một vòng tròn r = 0.75 m, đường kính 1.50 m.
Ngón cái / Thumb_Up	Xác nhận và bắt đầu lựa chọn hợp lệ.
Xòe tay / Open_Palm	Dừng ngay; không tự tiếp tục.
Nắm tay / Closed_Fist	Hủy hành vi, xóa lựa chọn, về IDLE.

Wave dùng thêm thuật toán theo thời gian. Tư thế xòe tay được xác nhận từ khớp AI hoặc nhãn Open_Palm ≥0.50 phù hợp với các ngón. Cổ tay dịch ≥0.12 chiều rộng ảnh, ≥2 lần đảo chiều trong 2.4 s, ≥6 mẫu; chịu được một frame không chắc chắn ngắn. Wave giữ khi cùng tay còn thấy để chuyển sang Thumb_Up xác nhận. Khi RUNNING, Wave tắt để Open_Palm luôn dừng robot.

3. Máy trạng thái và chống nhiễu



Hình 2. Mọi chuyển động, kể cả xoay chào, đều cần bước xác nhận.

Chọn/xác nhận ổn định ≥0.30 s và ≥3 message; lựa chọn hết hạn 12 s. Khi SELECTED đứng yên, cho phép mất tay 1.8 s; nắm tay giữ 0.25 s để hủy. Khi RUNNING, Open_Palm/Closed_Fist dừng ngay, mất tín hiệu 0.65 s gây dừng. Chọn hành vi khác dừng chương trình cũ, về SELECTED và cần Thumb_Up mới. Giữ cùng cử chỉ không lặp hành vi. Sau lỗi có thể chọn lại cùng cử chỉ.

4. Topic, điều khiển và cơ chế dừng

Topic / kiểu message	Vai trò
/detected_gesture / String	Nhãn cử chỉ; Best Effort, Volatile, depth 1; không replay lệnh cũ.
/cmd_vel / Twist	linear.x là vận tốc tiến/lùi; angular.z là tốc độ quay.
/odom / Odometry	Độ dời và yaw thực tế từ mô phỏng.
/scan / LaserScan	Khoảng cách vật cản phía tiến, lùi và quanh robot.
/dock_status / DockStatus	Chỉ cho chạy khi robot không dock.
/hazard_detection / HazardDetectionVector	Khóa chuyển động khi có va chạm, cliff, stall, wheel drop hoặc proximity.
/behavior_state / String JSON	Mode, lựa chọn, phân đoạn, lý do dừng và trạng thái sẵn sàng.
/gesture_debug / String JSON	Nhãn AI, confidence và nhãn đã publish.

Tiến–lùi: hai đích cố định +0.50 m và 0 m theo tư thế lúc xác nhận; lặp 4 chu kỳ, tối đa 0.28 m/s. Mỗi chu kỳ trở về gần điểm đầu; giao diện ghi lượt 1/4…4/4 và TIẾN/LÙI. **Đi tròn:** r=0.75 m, v=0.26 m/s, ω≈0.347 rad/s; kiểm tra cả góc gần 2π và quãng đường. **Chào:** quay +π/4, −π/2, +π/4, tối đa 1.0 rad/s. Giảm tốc khi gần cuối đoạn.

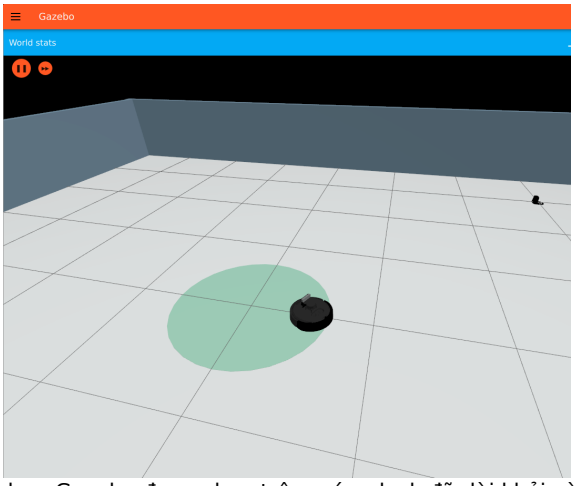
Đủ odometry mới hoàn thành; đứng yên không thể hoàn tất bằng thời gian trôi qua. Không tiến triển 12 s hoặc quá 180 s gây FAULT. prepare_arena xác minh driver Gazebo dùng use_sim_time=true rồi đặt backup_only để lùi đủ 0.50 m; không dùng full, các hazard khác và LiDAR phía sau vẫn hoạt động.

Dừng khi mất điều khiển: quá 0.65 s không có nhãn hợp lệ, node phát Twist 0. Kết quả AI chỉ hợp lệ khi frame còn mới ≤0.30 s; vòng điều khiển dùng steady clock 20 Hz. Cảm biến cũ quá 0.8 s, obstacle phía tiến/lùi <0.42 m, khoảng trống xoay <0.26 m hoặc hazard nguy hiểm cũng gây dừng. Driver có timeout cmd_vel riêng 0.5 s.

Các timeout là ngân sách phản ứng phần mềm; dừng vật lý còn phụ thuộc chu kỳ truyền, driver và hãm bánh xe. Không có cơ chế tự resume sau khi đưa tay trở lại. Hai thread capture/inference không được giữ heartbeat sống bằng một frame cũ vô thời hạn.

5. Môi trường và kết quả kiểm chứng

Arena 8×8 m, tường bao và vùng tâm trống. prepare_arena dời dock đến (3,3), kiểm tra undocked, không dời robot hoặc gửi vận tốc. Overlay bài 6 bỏ RGB-D mô phỏng không dùng; giữ webcam laptop, LiDAR và an toàn driver. Cấu hình đo: tiến/lùi 0.28 m/s, quay 1.0 rad/s, đi tròn 0.26 m/s.

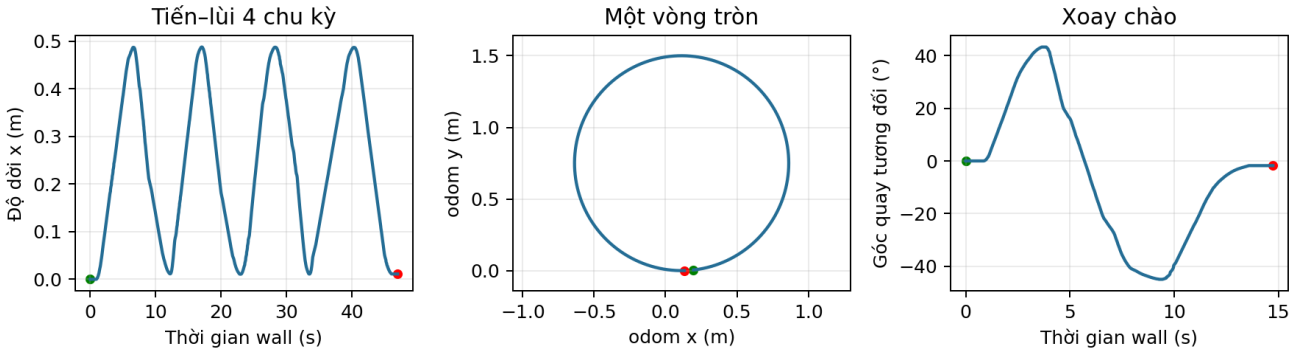


Hình 3. Ảnh chụp Gazebo đang chạy trên máy; dock đã dời khỏi vùng thao tác.

Kiểm thử logic: 52 passed in 0.42s. Kiểm tra chọn–xác nhận, nhiễu nhãn, dừng/hủy, mất tín hiệu/cảm biến, vật cản, toàn bộ các hành vi, unwrap yaw và phát hiện Wave.

Kiểm thử ROS / Gazebo với nhãn String tự động	Kết quả
Tiến–lùi đủ 4 chu kỳ	Đạt
Đi đủ một vòng tròn	Đạt
Xoay chào đủ 3 đoạn	Đạt
Open_Palm dừng	Đạt
Closed_Fist hủy	Đạt
Mất tín hiệu dừng	Đạt

ODOMETRY THẬT — đầu vào String tự động, không phải demo webcam



Hình 4. Quỹ đạo đo từ /odom của lần kiểm thử được lưu trong artifacts.

Giới hạn bằng chứng: các chuyển động trên dùng nhãn String tự động để kiểm tra ROS/Gazebo. Webcam và model đã chạy inference trên ảnh thật; chưa có video người thật thực hiện đủ các cử chỉ. Không coi log tự động hoặc sơ đồ là bằng chứng nhận diện tay thành công.

6. Chạy chương trình và quay video

Cài đặt và build theo README.md. Tạo venv Python 3.10 có system-site-packages, cài requirements.txt, tải model bằng tools/download_model.py. Mọi terminal phải source ROS, install/setup.bash và scripts/lab_env.bash để dùng chung ROS_DOMAIN_ID và middleware.

Bước	Lệnh / thao tác
1. Simulation	ros2 launch turtlebot4_gesture_hri simulation.launch.py
2. Chuẩn bị scene	ros2 run turtlebot4_gesture_hri prepare_arena
3. Hai node + webcam	ros2 launch turtlebot4_gesture_hri gesture.launch.py python_executable:=<venv>/bin/python camera_index:=0
4. Quan sát	Webcam hiển thị nhãn/confidence, robot mode; Gazebo hiển thị robot thực hiện hành vi.

Kịch bản đầy đủ ở docs/DEMO_CHECKLIST.md. Quay đồng thời webcam và Gazebo: chọn không chạy → Thumb_Up mới chạy → 4 chu kỳ tiến/lùi → đi tròn → xoay chào → Open_Palm dừng → Closed_Fist hủy → cất tay gây dừng. Giữ một tay trong webcam khi đang chạy; giữ nguyên Thumb_Up sau hoàn tất để minh họa không lặp hành vi. Không dùng phím điều khiển robot hoặc gửi cmd_vel thủ công trong demo.

7. Liên kết nộp bài

GitHub Public: CHỜ BỔ SUNG URL THẬT.

Video Google Drive Public: CHỜ BỔ SUNG URL THẬT.

Repository chứa source package, launch, config, world, README và báo cáo. Không đưa venv/build/install/log hoặc model weights lên GitHub. Video cần upload Drive và bật Anyone with the link / Viewer; kiểm tra liên kết bằng cửa sổ riêng tư. Xuất PDF lại sau khi điền các URL thật.

8. Tài liệu tham khảo chính thức

MediaPipe Gesture Recognizer Python

MediaPipe model và danh mục cử chỉ

OpenCV đọc webcam

TurtleBot 4 điều khiển vận tốc

Create 3 safety và backup limit